

PROJECT PHASE 2

Team Name : NotAGI

Team Members:

NAME	ASU ID
ABISHEK PRAKASH NAGARAM	1237484835
AARYA BHUTKAR	1237397982
SASWATA DUTTA	1237229450
BENJAMIN ASPINALL	1211505565
NISHANT HIRE	1237584610

TYPES OF ERRORS FOUND UPON ERROR ANALYSIS

Based on a manual analysis of conversation trajectories, four primary error categories cause language agents to fail in dynamic environments: (**REFERENCE** - MAST DATASET)

1. TYPE 1 - SYSTEM DESIGN ISSUES

- Inadequate Tooling:** This occurs when the agents lack the specific tools required to solve the assigned task.
- Agent Role Misconfiguration:** This involves giving agents roles or descriptions that do not match the skills needed for the task.
- Instruction Unclear:** The initial prompt or task description is too vague for the agents to execute correctly.

2. TYPE 2 - INTER-AGENT MISALIGNMENT

- Communication Overhead:** Agents spend excessive time communicating without making actual progress on the task.
- Information Siloing:** A failure where an agent possesses critical information but fails to share it with the rest of the system
- Chain-of-Command Failure:** Agents ignore instructions from a "lead" agent or fail to follow the intended workflow.

3. TYPE 3 - TASK VERIFICATION FAILURES

- a. **Premature Termination:** The system stops working before the goal is reached, often because an agent falsely claims a task is finished
- b. **Incorrect Verification:** An agent checks another agent's work but incorrectly identifies it as correct.
- c. **Goal Drift:** The system slowly moves away from the original user intent over multiple turns of conversation.
- d. **Infinite Loop:** Agents repeat the same incorrect actions or dialogue indefinitely without identifying the error.

SET OF ERRORS CATEGORIES

1. USER INSTRUCTION HALLUCINATION

This corresponds to **Goal Drift & Instruction Unclear**.

- **Example from Trace (Task ID 29):** The agent unnecessarily addressed a garden hose exchange when the user's explicit instruction and the required output were focused solely on presenting options and prices for a skateboard exchange.
- **Reasoning:** This is a case of **Goal Drift** because the agent expanded the scope of the task beyond the intended goal, deviating from the functional requirement to report specific numeric values (skateboard prices) by involving unrelated order items.

2. AGENT HALLUCINATION

This corresponds to **Incorrect Verification & Premature Termination**

- **Example from Trace (Task ID 73):** The agent incorrectly fabricated order and item IDs (specifically using `order_id "78901"` and `item_ids [101-120]`) instead of utilizing the actual ground truth values provided in the system (`#W5272531` and `7228247242`).
- **Reasoning:** This is a clear case of **Agent Hallucination** because the agent bypassed the retrieval of real data and substituted it with hallucinated placeholders to complete the action call. This satisfies the criteria of providing an incomplete or incorrect response that does not actually satisfy the data needs of the transaction.

3. DOMAIN POLICY VIOLATION

This corresponds to **Chain-of-Command Failure**

- **Example from Trace (Task ID 11):** The agent failed to follow the user's explicit instruction to refund each order to the **opposite** payment method (e.g., swapping gift card for credit card), instead defaulting to the original payment methods.
- **Reasoning:** This constitutes a **Domain Policy Violation** because the agent ignored a high-level "business rule" defined for the transaction, failing to execute the specific refund logic required by the user's instruction.

4. CONTEXTUAL MISINTERPRETATION

This corresponds to Agent Role Misconfiguration & Conflict in Logic.

- **Example from Trace (Task ID 85):** The agent incorrectly invoked the `exchange_delivered_order_items` tool (which is for delivered orders) when the ground truth required `modify_pending_order_items` (which is for pending orders).
- **Reasoning:** This is a **Contextual Misinterpretation** because the agent failed to accurately identify the order's state (pending vs. delivered). This led to a functional mismatch where the agent applied an exchange protocol to an item that was still eligible for modification.

MULTI AGENT SYSTEM FRAMEWORK

DOMAIN POLICY VIOLATIONS

- LLMs struggle with "context drift" in long conversations. In complex tasks like those in tau-bench, an agent might successfully complete a user's request but violate a subtle domain rule
- The solution is a **Policy-Sentinel Architecture**. It is a secondary layer of reasoning that intercepts a planned action and verifies it against a formal constraint set and the current system state.
- It uses a specialized agent (The Sentinel) to extract relevant rules from a database, query the current environment (e.g., database status), and provide a "Go/No-Go" decision based on a logical comparison of the two.
- This intervention must occur **immediately after** the primary agent formulates a tool call but **before** that call is executed in the real-world or production environment.

The Multi-Agent Sentinel (The "Reviewer" Approach)

Implementation: Create a "Critic" or "Reviewer" agent whose only job is to find reasons *not* to allow an action.

The Workflow:

1. **Agent A** proposes: `exchange_item(order_id="#W2890441", items=["chair"])`.
2. **Sentinel Agent** queries the database: `get_order_status("#W2890441")`.
3. **Database** returns: `status="return_requested"`.
4. **Sentinel compares to Policy:** "Orders must be in 'delivered' status to be exchanged".
5. **Result:** The Sentinel blocks the call and sends a message to the agent: "Correction needed: Order is already in return status".

CONTEXTUAL MISINTERPRETATION

- LLMs often fail to grasp the nuance or implicit intent behind a user's request. For example, a user asking to "get a new item instead of the broken one" might be misinterpreted as a simple "return" rather than a complex "exchange," leading the agent to call the wrong tool
- This solution is a **Semantic Workflow Mapping Layer**. It acts as a cognitive bridge that translates high-level natural language into a structured, multi-step execution plan based on pre-defined business logic.
- It uses a **Tool Selector Agent** to suggest relevant tools and encodes human-like expertise into **Standardized Operating Procedures (SOPs)** that define the specific documents and verification steps required for any given intent.
- This occurs during the **Input Reformulation** stage, before the agent begins its reasoning chain, to ensure the entire trajectory is grounded in the correct domain context.

THE IMPLEMENTATION: THREE-LAYER WORKFLOW

Layer 1: Intent Canonicalization (The "Standardizer")

Instead of letting the LLM guess the intent, we use a "Canonicalization" step.

- **Technical Implementation:** A dedicated agent maps raw user text to a formal **Intent Schema** (e.g., "get a new one" -> **INTENT_EXCHANGE**).
- **Reference:** This addresses the "Canonicalization" challenges identified in the **ToolSandbox** benchmark.

Layer 2: Document-Driven SOPs (The "MetaGPT" Approach)

Once an intent is canonicalized, the system triggers a specific **SOP**.

- **Technical Implementation:** The system generates a **Requirement Document** that explicitly lists the necessary information for that intent (e.g., for an exchange, it *must* have the **original_order_id** and the **new_item_id**).
- **Benefit:** This eliminates "hallucinated progress" by requiring agents to produce structured artifacts like PRDs and System Designs before code or tool invocation.

Layer 3: Cognitive Planning & Tool Suggestion (The "IRMA" Approach)

The system reformulates the input with specific tool suggestions and memory tags.

- **Technical Implementation:**
 1. **Memory Module:** Retains the initial request to prevent "context drift" in long turns.

2. **Tool Selector Agent:** Provides a short list of relevant tools (e.g., `search_direct_flight` vs `search_onestep_flight`) with one-line explanations to help the agent disambiguate similar tools.

USER INSTRUCTION HALLUCINATION

- LLM-simulated users (and sometimes real users) suffer from **context drift** over long conversations. As the dialogue progresses, the model increasingly relies on recent turns, leading it to "forget" the initial goal or follow high-probability but incorrect continuations.
- This solution is a **Stateful Instruction Memory & Verification Layer**. It is a mechanism that mirrors the original mission statement in every prompt to ensure the agent doesn't deviate from the "gold standard" task.
- It uses a dedicated **Memorization Module** to store queries throughout the trajectory and a **FACT (Follow-up Question Acting) Agent** to detect when current inputs contradict the original instruction.
- This must be active during **every turn reformulation**. Before the assistant sees the latest message, the system injects the historical "source of truth" into the current context window.

THE IMPLEMENTATION: THREE-LAYER WORKFLOW

Layer 1: The Instruction Vault (Persistent Memory)

Instead of letting the conversation history flow as an unstructured string, we use the IRMA Memorization Module.

- **Technical Implementation:** The original user query is wrapped in `<memory>` tags and appended to every prompt.
- **Optimization:** To solve the "Lost in the Middle" problem (where models ignore info in the center of long prompts), the system places this `<memory>` block at the very top or bottom of the current context window to leverage Primacy and Recency biases.

Layer 2: Discrepancy Detection (The FACT Agent)

The system employs the **FACT (Follow-up Question ACTing)** logic to audit incoming messages.

- **Technical Implementation:** Before responding, the model is instructed to:
 1. Compare the new user input against the `<memory>` vault.
 2. **Identify Data Conflicts:** If the user previously asked for a "refund for 3" but now says "refund for 2," the agent is forbidden from calling a tool.
 3. **Mandatory Clarification:** The agent *must* ask a follow-up question to resolve the conflict before any state change occurs.

Layer 3: Human-in-the-Loop (HITL) Handoff

For unresolvable hallucinations in complex simulations, the system uses a fail-safe trigger.

- **Technical Implementation:** If the **FACT** agent detects that the "simulated user" is consistently providing information that contradicts the "domain rules" or "original script," it is programmed to pause the interaction and hand it off to a human supervisor to prevent a system-wide failure.

AGENT HALLUCINATION

- Agents often suffer from "Hallucinated Progress," where they claim to have executed a command (e.g., "I have updated the file") when no action was actually taken. They also exhibit "Premature Termination," stopping the workflow before the final goal is reached, often due to getting stuck in loops or incorrectly triggering a "finished" token.
- This solution is an **Automated Task-Verification Framework**. It is a cross-check system where a secondary agent or a rule-based validator inspects the environment to confirm that the intended state change actually occurred.
- It uses **Milestone-based Key Performance Indicators (KPIs)** to track sub-goals, **Outcome-Based Rewards** to penalize false claims, and a **QA/Verifier Agent** to sign off on every completed step.
- This must occur **immediately after an agent reports completion** of a sub-task or the final goal, and **before** the system delivers a final response to the user.

Technical Implementation: The Verification

Layer 1: Milestone-Based KPI Tracking (The Progress Auditor)

Instead of a binary pass/fail, the system uses "Milestones" to measure objective progress.

- **Technical Implementation:** The original instruction is decomposed into a list of required **Execution Milestones** (e.g., **Milestone 1: Retrieve User ID**, **Milestone 2: Authenticate**, **Milestone 3: Process Refund**).
- **Novelty:** The system uses a **Cognitive Planning** strategy to check these milestones in real-time. If an agent tries to terminate the conversation before all milestones are marked as "Verified," the **QA Agent** interrupts and forces a re-plan.

Layer 2: Outcome-Driven Validation (The ReTool Method)

This layer ignores the agent's "Thought" process and only looks at the "Result."

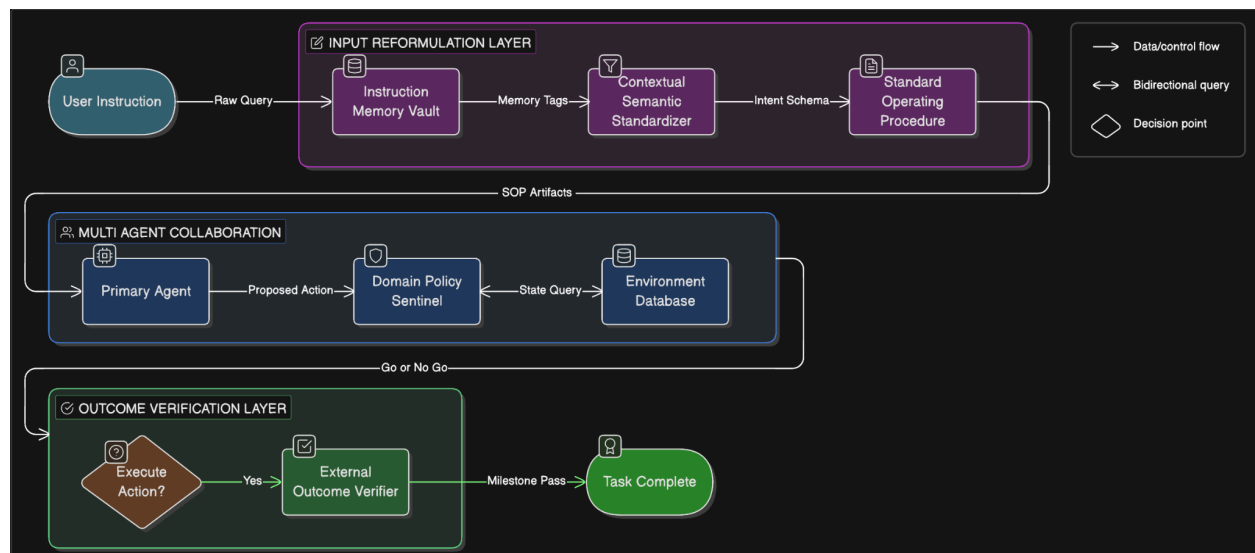
- **Technical Implementation:** The system employs a **Rule-Based Outcome Reward**. After an agent claims to have called a tool, a script performs an **Environment Audit** (e.g., checking if the database record was actually updated).
- **The "Aha Moment":** If the audit fails (e.g., a "Hallucinated Progress" error), the system feeds the failure back to the agent as **Interpreter Feedback**. This triggers a "self-correction" behavior where the agent realizes its mistake and re-attempts the action with correct logic.

Layer 3: Role-Based Verification (The MetaGPT SOP)

The system incorporates **Standardized Operating Procedures (SOPs)** that mandate a "Reviewer" role.

- **Technical Implementation:** In an "Assembly Line" paradigm, the **Engineer Agent** writes the code or tool-call, but the **QA Agent** must execute it in a sandbox and verify the output.
- **Constraint Enforcement:** The system is programmed so that the "Project Manager" agent cannot deliver the final "Success" message to the user until the QA Agent provides a **Verification Document** confirming the environment state matches the goal.

FRAMEWORK DIAGRAM



REFERENCES

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. 2025. Why do multi-agent llm systems fail? arXiv preprint arXiv:2503.13657.

Guoxin Chen, Zhong Zhang, Xin Cong, Fangda Guo, Yesai Wu, Yankai Lin, Wenzheng Feng, and Yasheng Wang. 2024b. Learning evolving tools for large language models. arXiv preprint arXiv:2410.06617.

Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. 2025. Retool: Reinforcement learning for strategic tool use in llms. arXiv preprint arXiv:2504.11536.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. arXiv preprint arXiv:2304.08244.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. 2024. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. arXiv preprint arXiv:2408.04682.

Mihir Parmar, Xin Liu, Palash Goyal, Yanfei Chen, Long Le, Swaroop Mishra, Hossein Mobahi, Jindong Gu, Zifeng Wang, Hootan Nakhost, et al. 2025. Plangen: A multi-agent framework for generating planning and reasoning trajectories for complex problem solving. arXiv preprint arXiv:2502.16111.

Akshara Prabhakar, Zuxin Liu, Weiran Yao, Jianguo Zhang, Ming Zhu, Shiyu Wang, Zhiwei Liu, Tulika Awalgaonkar, Haolin Chen, Thai Hoang, et al. 2025. Apigen-mt: Agentic pipeline for multi-turn data generation via simulated agent-human interplay. arXiv preprint arXiv:2504.03601.

Lianlei Shan, Shixian Luo, Zezhou Zhu, Yu Yuan, and Yong Wu. 2025. Cognitive memory in large language models. arXiv preprint arXiv:2504.02441.

Joykirat Singh, Raghav Magazine, Yash Pandya, and Akshay Nambi. 2025. Agentic reasoning and tool integration for llms via reinforcement learning. arXiv preprint arXiv:2505.01441.

Fan Yin, Zifeng Wang, I Hsu, Jun Yan, Ke Jiang, Yanfei Chen, Jindong Gu, Long T Le, Kai-Wei Chang, ChenYu Lee, et al. 2025. Magnet: Multi-turn tool-use data synthesis and distillation via graph translation. arXiv preprint arXiv:2503.07826.

Shaokun Zhang, Yi Dong, Jieyu Zhang, Jan Kautz, Bryan Catanzaro, Andrew Tao, Qingyun Wu, Zhiding Yu, and Guilin Liu. 2025. Nemotron-research-tool1: Exploring tool-using language models with reinforced reasoning. arXiv preprint arXiv:2505.00024.